

Overview

The purpose of the assignment is to assess object oriented analysis and modelling skills, Java coding skills, and code structuring. Take your time on the task, but don't get too carried away. If you submit a solution that is in any way incomplete, the parts that you decided to focus on are relevant.

Keeping the objective in mind, you are free to use whatever tools, libraries, frameworks at your disposal. Artificial Intelligence (AI) usage is encouraged. Please include a README in any format on how to run and use the program.

Requirements

Your task is to write a backend application that will simulate sports betting event outcome handling and bet settlement via Kafka and RocketMQ.

- **An API endpoint to publish a sports event outcome to Kafka.**
- **A Kafka consumer that listens to `event-outcomes` Kafka named-topic.**
- **Matches the event outcome to bets that need to be settled.**
- **A RocketMQ producer that sends messages to `bet-settlements`.**
- **A RocketMQ Consumer that listens to `bet-settlements`.**

The details of these Use Cases can be found in the next page. There are no further defined requirements for the API, it is up to you to design and implement the necessary code in order for the API to support the mentioned operations above. This refers to the entire flow starting from the API, down to the persistence layer.

It is expected for you to spend around 90 minutes to complete the exercise.

Delivery

- The solution needs to be 100% executable
- Provide a link to the GitHub repository where your solution is committed
 - Please make sure it is not private or restricted
- Provide a README file (documentation) how to run and use the solution

Sports Betting Settlement Trigger Service

We want to launch a new backend service that will simulate sports betting event outcome handling and bet settlement via Kafka and RocketMQ. Here you have the relevant Use Cases to support:

1. An API endpoint to publish a sports event outcome to Kafka.

- a. This endpoint should publish an event outcome to Kafka. An event outcome is represented by:
 - i. Event ID
 - ii. Event Name
 - iii. Event Winner ID
- b. The topic should be `event-outcomes` as commented before.

2. A Kafka consumer that listens to `event-outcomes` Kafka named-topic.

3. Matches the event outcome to bets that need to be settled.

- a. The System checks if we have bets in our database that can be settled based on Event ID from the event outcome message.
- b. A bet in database is composed by:
 - i. Bet ID
 - ii. User ID
 - iii. Event ID
 - iv. Event Market ID
 - v. Event Winner ID
 - vi. Bet Amount

4. A RocketMQ producer that sends messages to `bet-settlements`.

- a. After identifying the bets to be settled in the previous point, here we should produce the messages for RocketMQ so the bet can be settled.

5. A RocketMQ consumer that listens to `bet-settlements`.

- a. Here we should consume the messages and settle the bets.

Conditions:

- If the RocketMQ setup is too complex, use mocks for the RocketMQ producer. Just log the payload.
 - Use an in-memory database for the bets.
-

Extensions

Below you can find sample task extensions we can ask the candidate during the BE technical interview. They should not take more than 15min each:

- Imagine we found out sometimes we have RocketMQ failures. Implement a fallback mechanism for RocketMQ failed delivery.
- Replace in-memory database by SQL one.
- Imagine now event outcomes for Kafka include multiple outcomes for the same event. For example, now the structure for the event outcome is like below. Modify your Kafka producer and Kafka consumer.
 - Event ID
 - Event Outcomes:
 - Outcome 1
 - Event Market ID
 - Winner ID
 - Outcome 2
 - ...

Reasoning (internal discussion)

I think the task can be implemented in 90 minutes because:

- 30 minutes to create basic app structure inside the IDE and the RocketMQ and Kafka setup.
- 15 minutes to implement the API endpoint to publish to Kafka.
- 20 minutes to implement the Kafka consumer to process events.
- 20 minutes to implement the RocketMQ producer.
- 5 additional minutes for cleanup and review -> 90 minutes in total.

This task requires the candidate to know what Kafka and RocketMQ are.